

T Deekshith

192472253

CSA6501- Generative AI

LAB EXPERIMENTS(1- 20)

1. Install Matplotlib and display a simple line graph.

```
colab GEN-AI

! pip install matplotlib

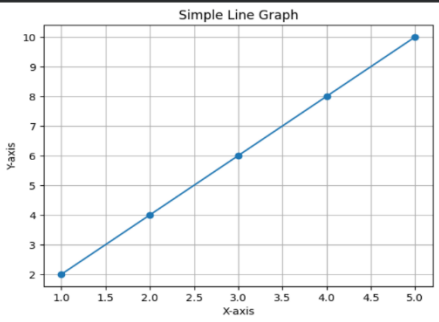
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

```
colab GEN-AI

! import matplotlib.pyplot as plt

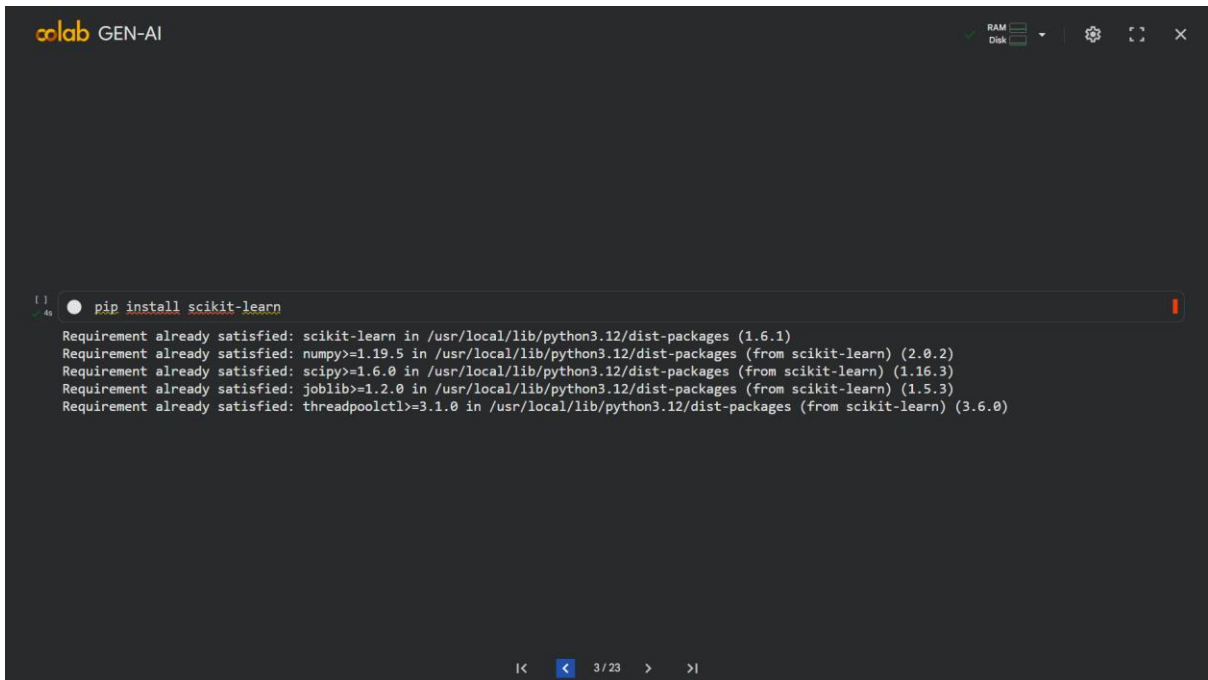
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plt.plot(x, y, marker='o')
plt.title("Simple Line Graph")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```



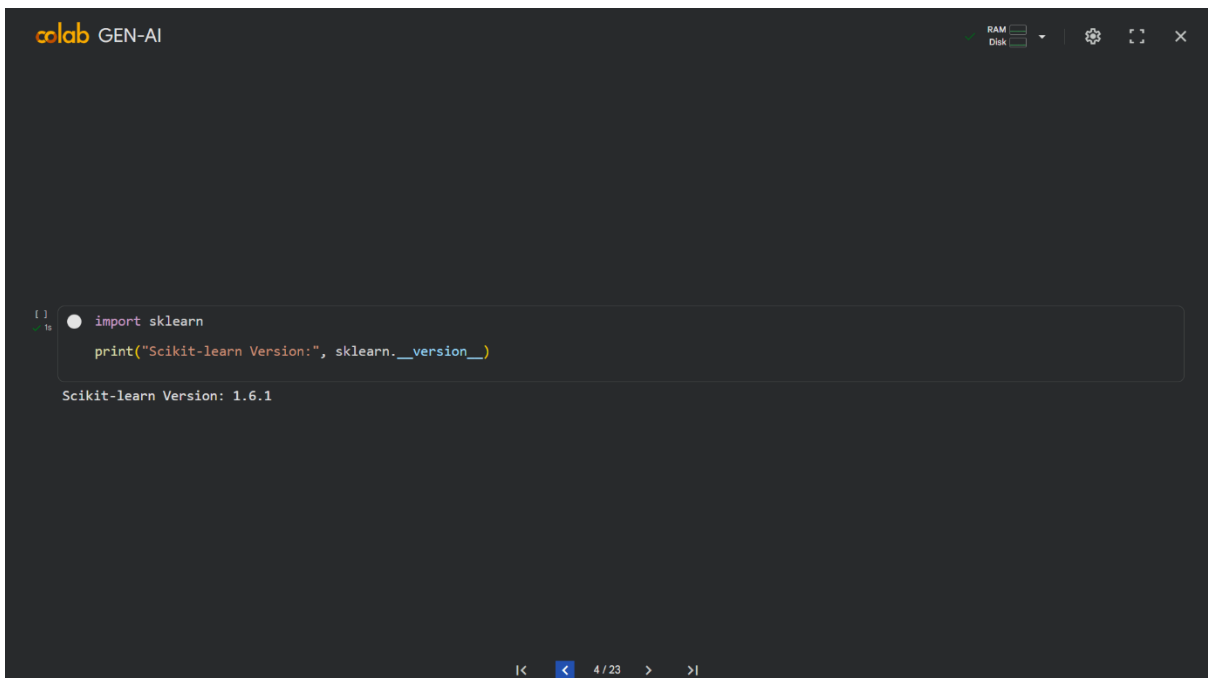
X-axis	Y-axis
1.0	2
2.0	4
3.0	6
4.0	8
5.0	10

2. Install Scikit-learn and print its version.



The image shows a Google Colab notebook interface with a dark theme. The top bar includes the 'colab GEN-AI' logo on the left and system status icons (RAM, Disk, settings, and window controls) on the right. A code cell is active, showing the command `pip install scikit-learn` and its output. The output lists several requirements that are already satisfied, including scikit-learn (1.6.1), numpy (2.0.2), scipy (1.16.3), joblib (1.5.3), and threadpoolctl (3.6.0). The bottom of the interface shows navigation icons and a page number of 3/23.

```
[1] • pip install scikit-learn
4s
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
```

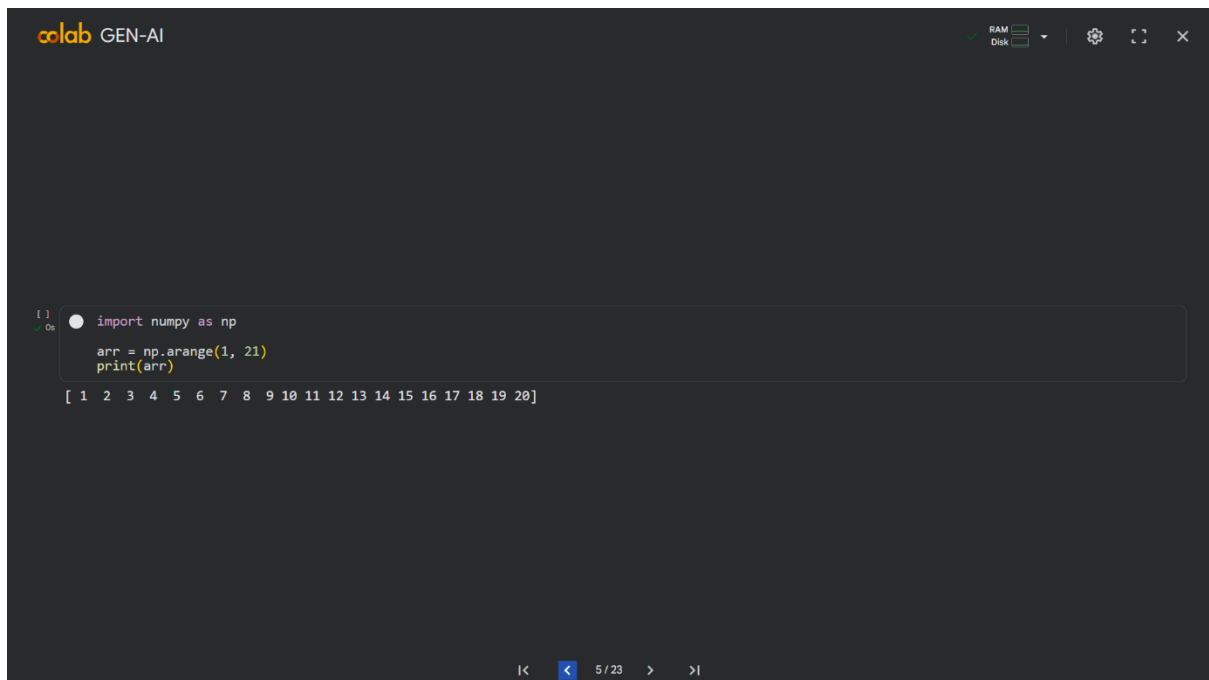


The image shows the same Google Colab notebook interface. A second code cell is active, showing the command `import sklearn` followed by `print("Scikit-learn Version:", sklearn.__version__)`. The output of the cell is `Scikit-learn Version: 1.6.1`. The bottom of the interface shows navigation icons and a page number of 4/23.

```
[1] • import sklearn
1s
print("Scikit-learn Version:", sklearn.__version__)

Scikit-learn Version: 1.6.1
```

3. Create a NumPy array containing the first 20 natural numbers.



The image shows a Colab GEN-AI interface with a dark theme. At the top left is the 'colab GEN-AI' logo. At the top right are icons for RAM, Disk, settings, and window management. A code cell is active, containing the following Python code:

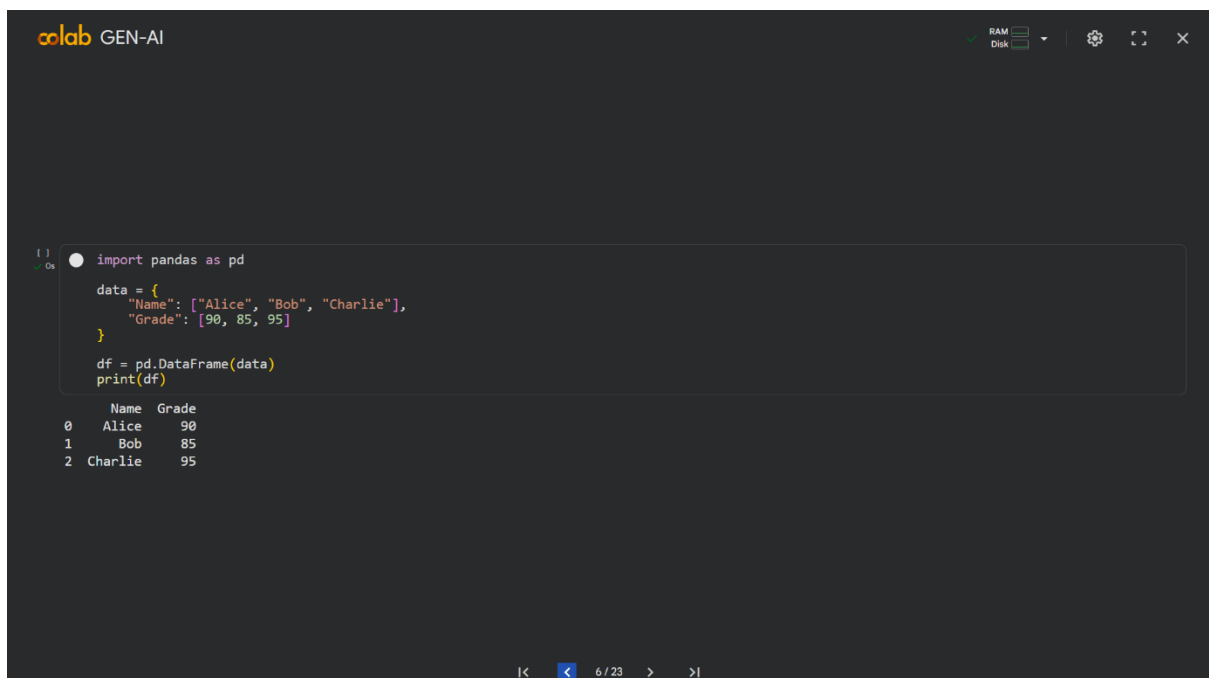
```
import numpy as np
arr = np.arange(1, 21)
print(arr)
```

The output of the code is displayed below the cell:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
```

At the bottom of the interface, there are navigation arrows and a page indicator '5 / 23'.

4. Create a Pandas DataFrame with student names and grades.



The image shows a Colab GEN-AI interface with a dark theme. At the top left is the 'colab GEN-AI' logo. At the top right are icons for RAM, Disk, settings, and window management. A code cell is active, containing the following Python code:

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Grade": [90, 85, 95]
}

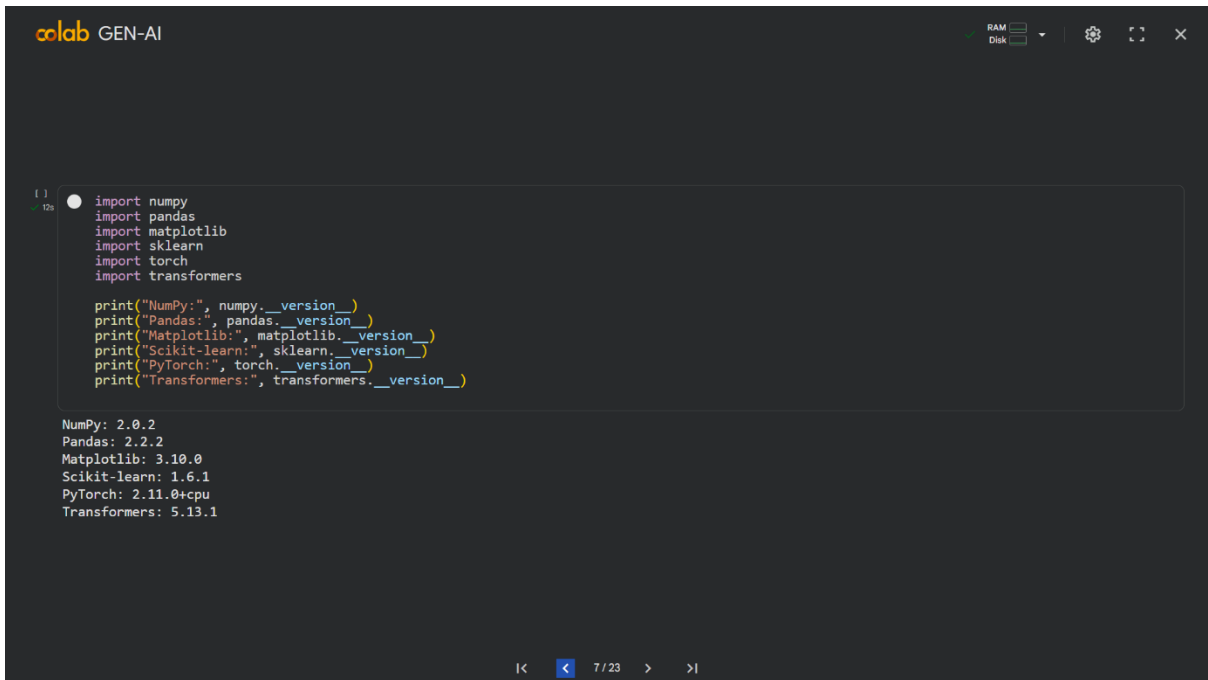
df = pd.DataFrame(data)
print(df)
```

The output of the code is displayed below the cell as a Pandas DataFrame:

	Name	Grade
0	Alice	90
1	Bob	85
2	Charlie	95

At the bottom of the interface, there are navigation arrows and a page indicator '6 / 23'.

5. Verify the installed versions of all AI libraries on your system.



```
import numpy
import pandas
import matplotlib
import sklearn
import torch
import transformers

print("NumPy:", numpy.__version__)
print("Pandas:", pandas.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Scikit-learn:", sklearn.__version__)
print("PyTorch:", torch.__version__)
print("Transformers:", transformers.__version__)
```

NumPy: 2.0.2
Pandas: 2.2.2
Matplotlib: 3.10.0
Scikit-learn: 1.6.1
PyTorch: 2.11.0+cpu
Transformers: 5.13.1

6. Create tensors using PyTorch and perform basic mathematical operations.



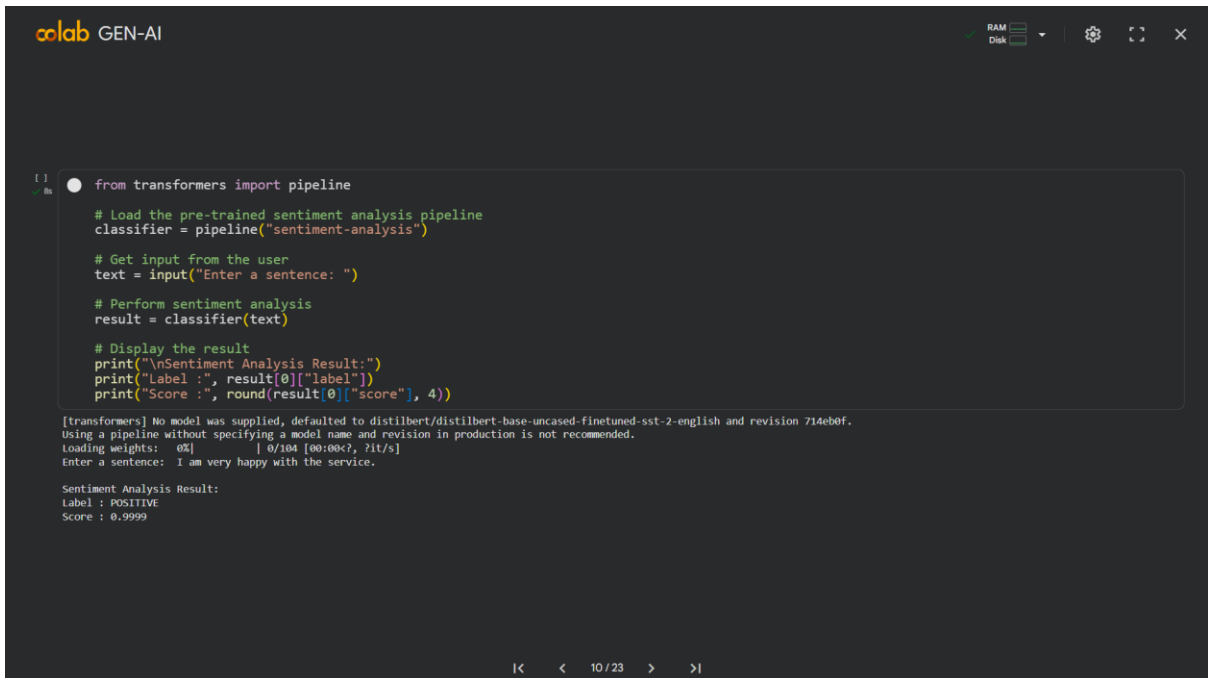
```
import torch

a = torch.tensor([1,2,3])
b = torch.tensor([4,5,6])

print("Addition:", a+b)
print("Multiplication:", a*b)
print("Dot Product:", torch.dot(a,b))
```

Addition: tensor([5, 7, 9])
Multiplication: tensor([4, 10, 18])
Dot Product: tensor(32)

7. Load a Pre-trained Transformer Model using Hugging Face and Perform Sentiment Analysis



```
[ ] In 10: from transformers import pipeline

# Load the pre-trained sentiment analysis pipeline
classifier = pipeline("sentiment-analysis")

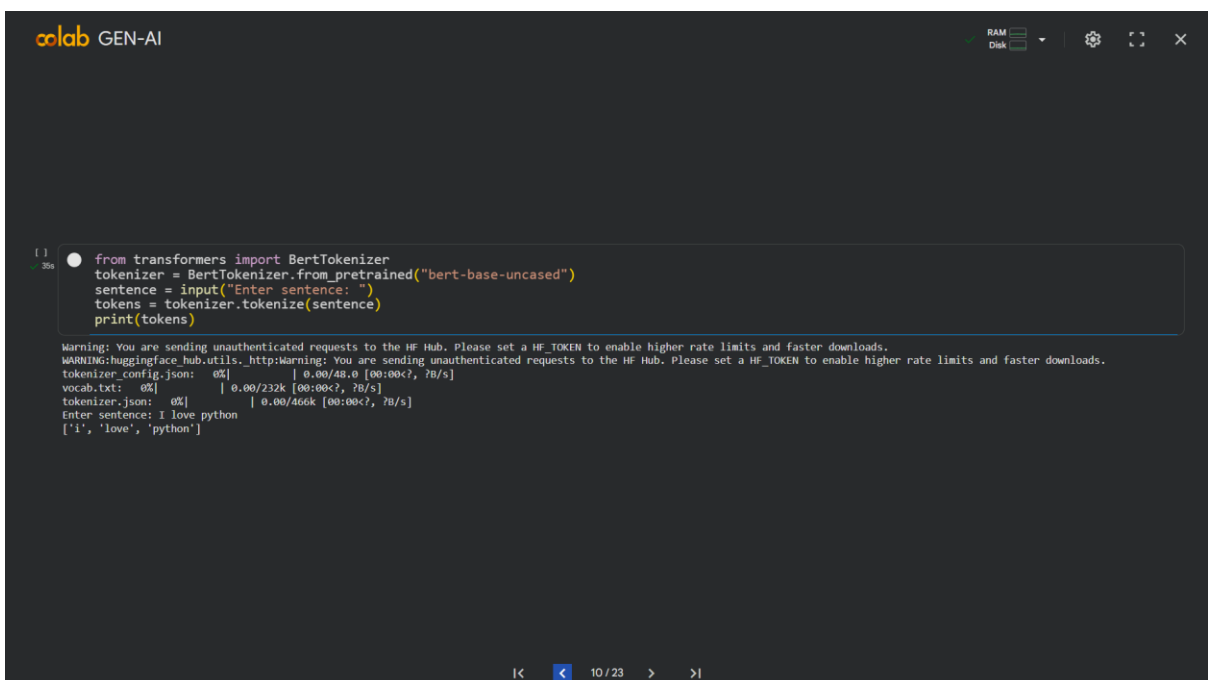
# Get input from the user
text = input("Enter a sentence: ")

# Perform sentiment analysis
result = classifier(text)

# Display the result
print("\nSentiment Analysis Result:")
print("Label :", result[0]["label"])
print("Score :", round(result[0]["score"], 4))
```

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f. Using a pipeline without specifying a model name and revision in production is not recommended.
Loading weights: 0% | 0/104 [00:00<?, ?it/s]
Enter a sentence: I am very happy with the service.
Sentiment Analysis Result:
Label : POSITIVE
Score : 0.9999

8. Tokenization using BERT Tokenizer



```
[ ] In 35: from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
sentence = input("Enter sentence: ")
tokens = tokenizer.tokenize(sentence)
print(tokens)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
tokenizer_config.json: 0% | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt: 0% | 0.00/232k [00:00<?, ?B/s]
tokenizer.json: 0% | 0.00/466k [00:00<?, ?B/s]
Enter sentence: I love python
['I', 'love', 'python']

9. Generate Text using GPT-2



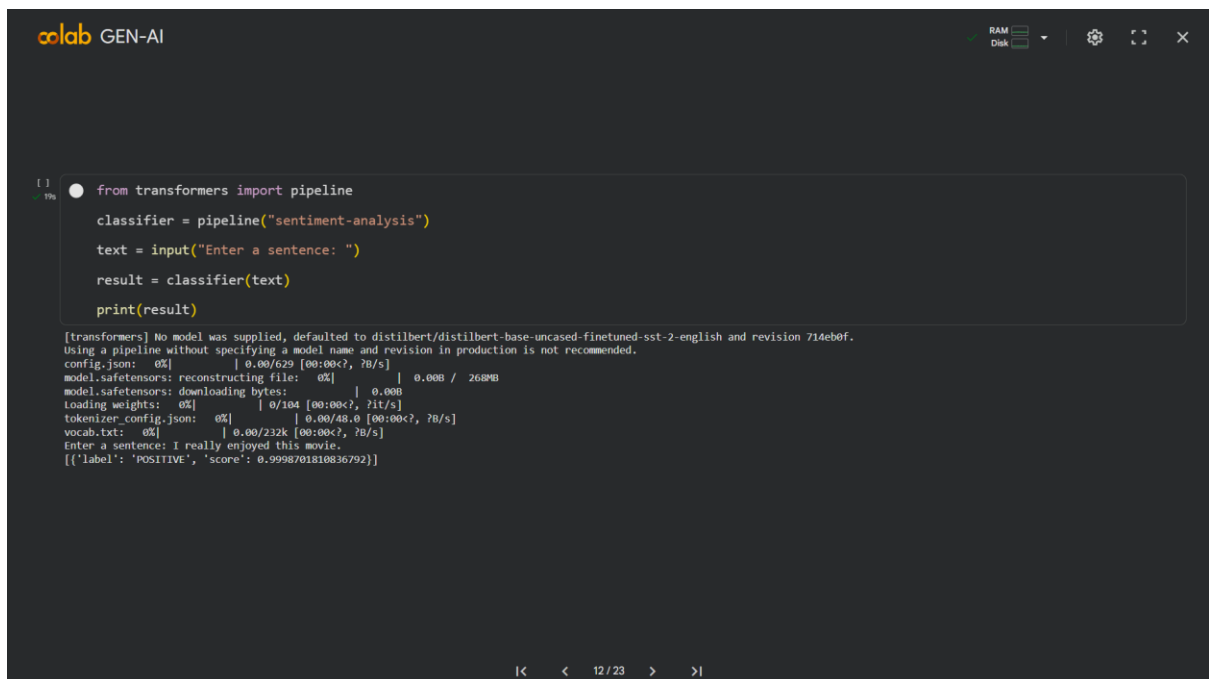
The Colab GEN-AI interface displays a Jupyter notebook cell for GPT-2 text generation. The code imports the pipeline, sets a prompt, and generates text. The output shows the loading progress of the GPT-2 model and the generated text snippet.

```
[1] # @title
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
prompt = input("Enter prompt: ")
result = generator(prompt, max_length=40)
print(result[0]["generated_text"])
```

Loading weights: 0% | 0/148 [00:00<?, ?it/s]
Enter prompt: Artificial Intelligence
[transformers] Both 'max_new_tokens' (=256) and 'max_length' (=40) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (h
Artificial Intelligence

A recent study found that artificial intelligence (AI) is a valuable tool for enhancing human creativity and creativity. The new study used a new combination of AI and machine learning t
In a paper published recently in the journal The Journal of Statistical Biology, researchers from Duke University's School of Computer Science and Science, along with researchers from th
The study found that researchers with the most advanced artificial intelligence techniques, including those from AI and machine learning, were more likely to create novel and novel ideas
"We found that researchers who were more intelligent than their counterparts were more likely to produce novel and novel ideas about how to improve human intelligence," said lead author
"These findings are important because they indicate that AI could be used to improve human intelligence, and could be used to improve the quality of human work."

10. Sentiment Analysis using Hugging Face Pipeline

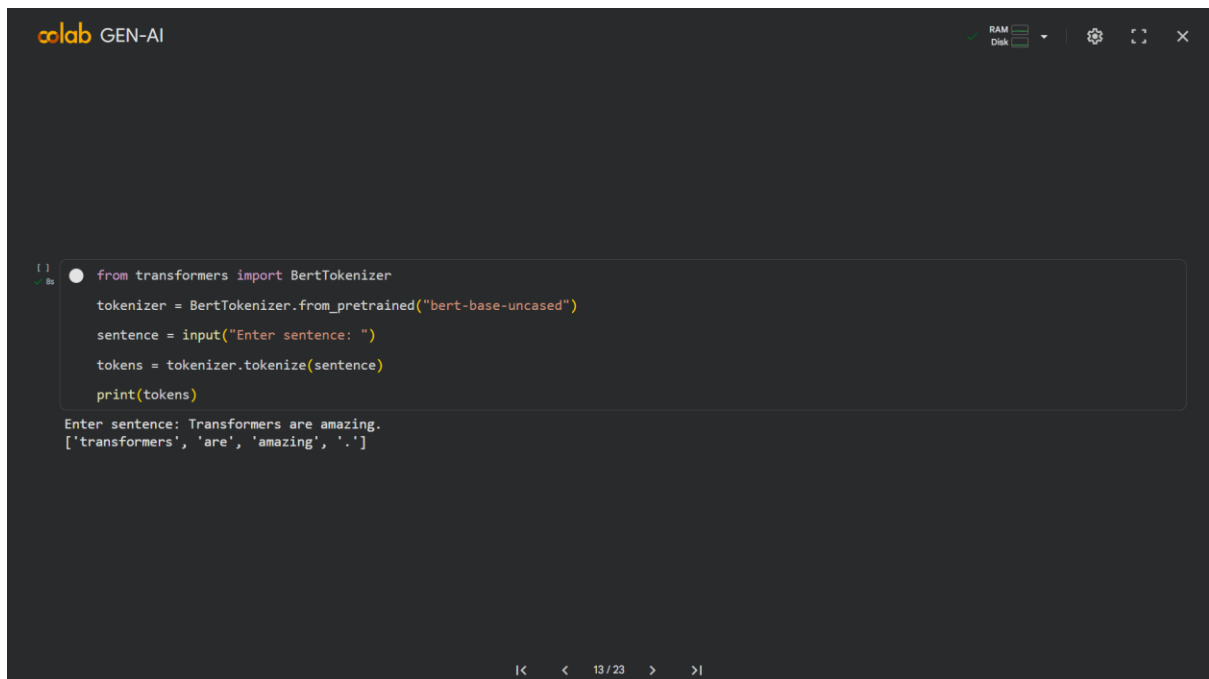


The Colab GEN-AI interface displays a Jupyter notebook cell for sentiment analysis using Hugging Face Pipeline. The code imports the pipeline, sets a sentence, and classifies the sentiment. The output shows the loading progress of the sentiment analysis model and the resulting sentiment label and score.

```
[1] from transformers import pipeline
classifier = pipeline("sentiment-analysis")
text = input("Enter a sentence: ")
result = classifier(text)
print(result)
```

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.
config.json: 0% | 0.00/629 [00:00<?, ?B/s]
model.safetensors: reconstructing file: 0% | 0.000 / 260MB
model.safetensors: downloading bytes: 0% | 0.000
loading weights: 0% | 0/104 [00:00<?, ?it/s]
tokenizer_config.json: 0% | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt: 0% | 0.00/232k [00:00<?, ?B/s]
Enter a sentence: I really enjoyed this movie.
[{'label': 'POSITIVE', 'score': 0.9998701810836792}]

11. Load BERT Tokenizer and Tokenize



The Colab GEN-AI interface displays a code cell with the following Python code:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = input("Enter sentence: ")

tokens = tokenizer.tokenize(sentence)

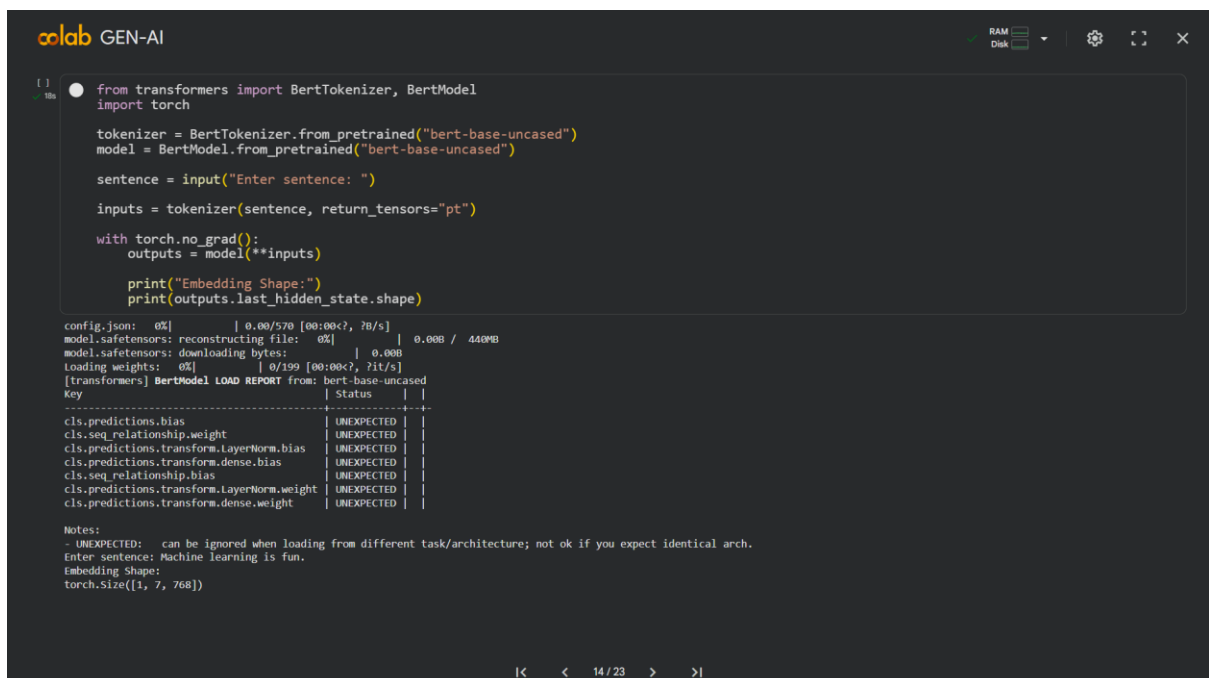
print(tokens)
```

The output of the code is:

```
Enter sentence: Transformers are amazing.
['transformers', 'are', 'amazing', '.']
```

The interface includes a top bar with the Colab logo and 'GEN-AI' text, and a bottom bar with navigation controls and a page indicator '13 / 23'.

12. Generate Contextual Embeddings using BERT



The Colab GEN-AI interface displays a code cell with the following Python code:

```
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

sentence = input("Enter sentence: ")

inputs = tokenizer(sentence, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

print("Embedding Shape:")
print(outputs.last_hidden_state.shape)
```

The output of the code is:

```
config.json: 0% | 0.00/570 [00:00<?, ?B/s] | 0.000 / 440MB
model.safetensors: reconstructing file: 0% | 0.000
model.safetensors: downloading bytes: 0% | 0.000
loading weights: 0% | 0/199 [00:00<?, ?it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----|-----|
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Enter sentence: Machine learning is fun.
Embedding Shape:
torch.Size([1, 7, 768])
```

The interface includes a top bar with the Colab logo and 'GEN-AI' text, and a bottom bar with navigation controls and a page indicator '14 / 23'.

13. Generate Text using GPT-2



```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = input("Enter prompt: ")

text = generator(prompt, max_length=60)

print(text[0]["generated_text"])
```

Loading weights: 0% | 0/148 [00:00<, ?it/s]

Enter prompt: Deep learning

[transformers] Both 'max_new_tokens' (~256) and 'max_length' (~60) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (h

Deep learning is a new field that has broad implications for computer science. This means the development of algorithms that understand neural network data - and how this relates to comp

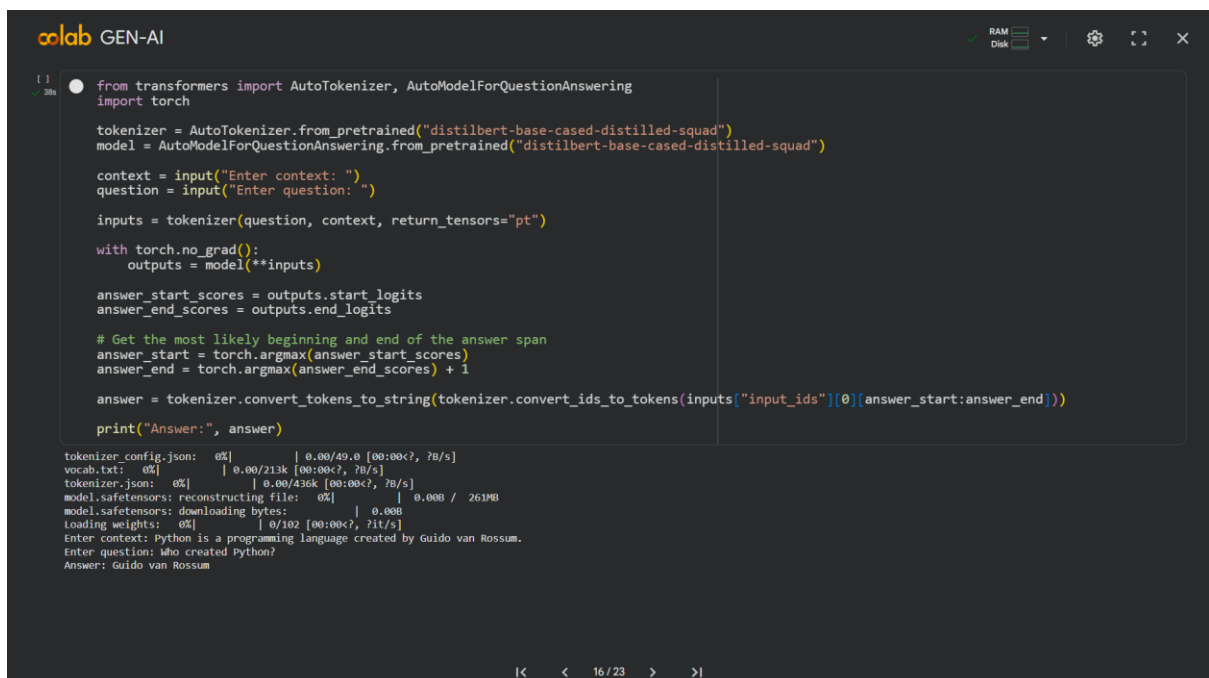
In an attempt to improve the efficiency of learning, researchers are developing new algorithms that are as efficient as ever.

In 2017, the Carnegie Mellon University Center for Machine Learning unveiled a system that can process more than 1,000,000 training-level training files, and is currently the top search

"The potential for these new applications is enormous," said Rob Schlesinger, a professor of computer science at Carnegie Mellon University, who led the research team.

"Most of the research focus on the ability to process highly-accurate training data, but many of the ideas that we have are applicable to other applications."

14. Question Answering using Pre-trained Transformer



```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-cased-distilled-squad")
model = AutoModelForQuestionAnswering.from_pretrained("distilbert-base-cased-distilled-squad")

context = input("Enter context: ")
question = input("Enter question: ")

inputs = tokenizer(question, context, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

answer_start_scores = outputs.start_logits
answer_end_scores = outputs.end_logits

# Get the most likely beginning and end of the answer span
answer_start = torch.argmax(answer_start_scores)
answer_end = torch.argmax(answer_end_scores) + 1

answer = tokenizer.convert_tokens_to_string(tokenizer.convert_ids_to_tokens(inputs["input_ids"][0][answer_start:answer_end]))

print("Answer:", answer)
```

tokenizer_config.json: 0% | 0.00/49.0 [00:00<, ?B/s]

vocab.txt: 0% | 0.00/213k [00:00<, ?B/s]

tokenizer.json: 0% | 0.00/436k [00:00<, ?B/s]

model.safetensors: reconstructing file: 0% | 0.000 / 261MB

model.safetensors: downloading bytes: | 0.000

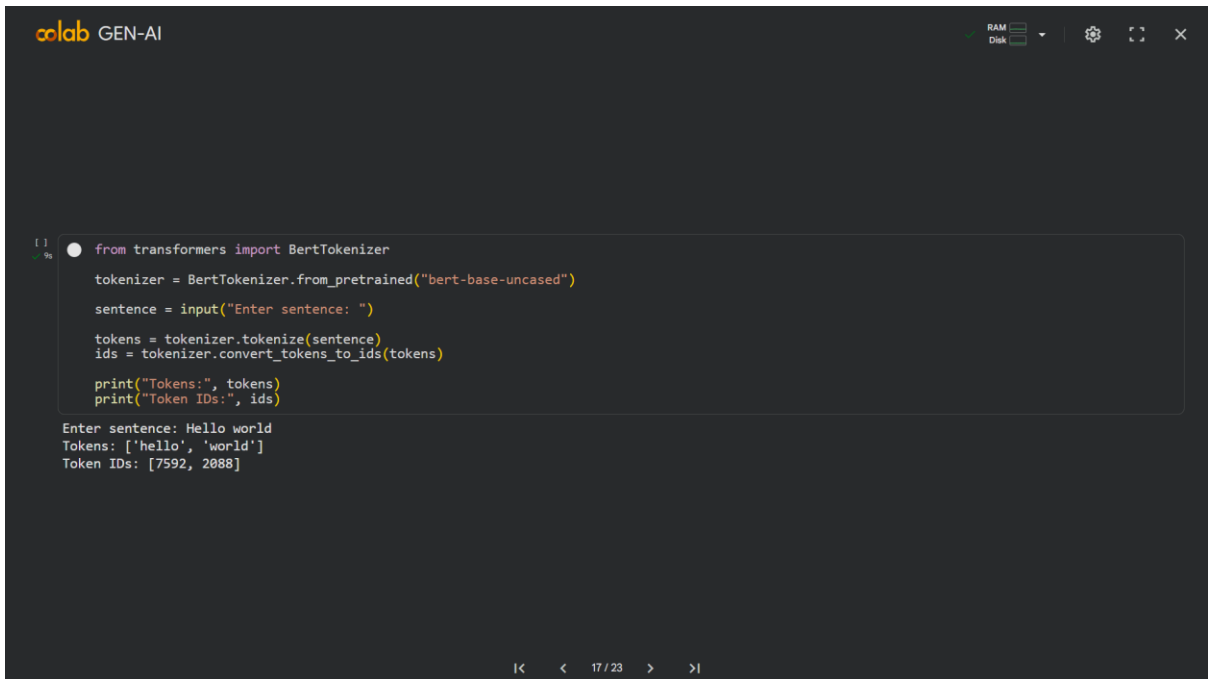
Loading weights: 0% | 0/102 [00:00<, ?it/s]

Enter context: Python is a programming language created by Guido van Rossum.

Enter question: Who created Python?

Answer: Guido van Rossum

15. Display Tokens and Token IDs



The image shows a Google Colab notebook titled "GEN-AI". The code cell contains the following Python code:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = input("Enter sentence: ")

tokens = tokenizer.tokenize(sentence)
ids = tokenizer.convert_tokens_to_ids(tokens)

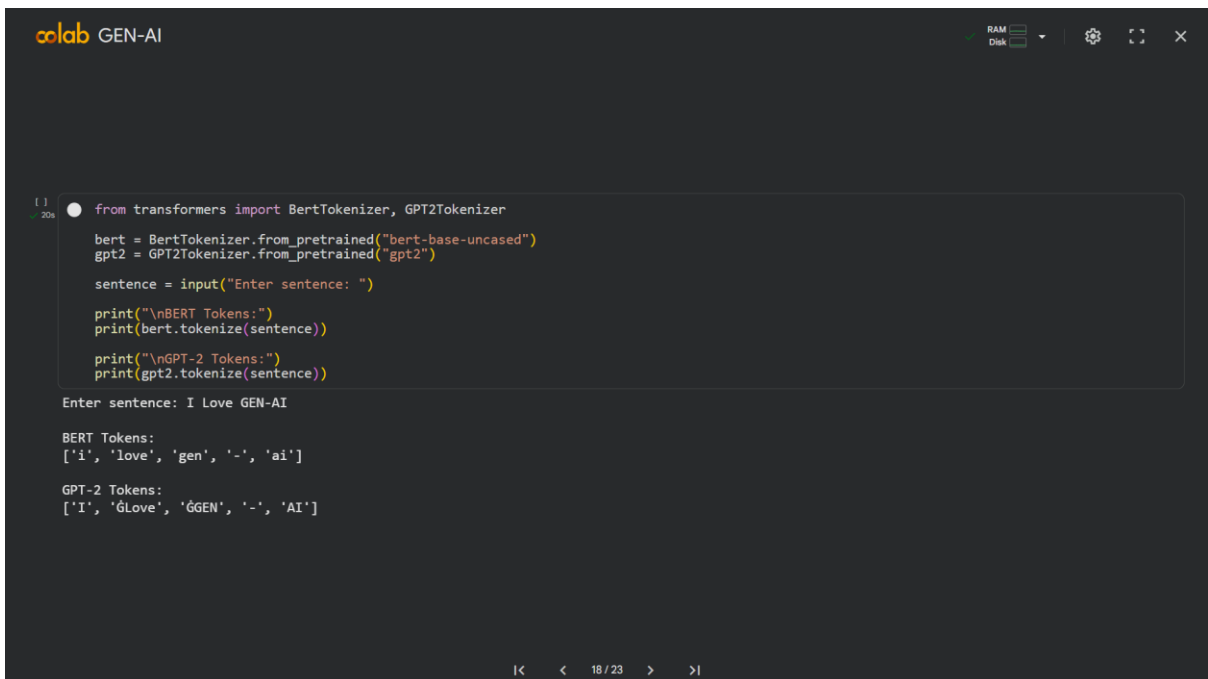
print("Tokens:", tokens)
print("Token IDs:", ids)
```

The output of the code is:

```
Enter sentence: Hello world
Tokens: ['hello', 'world']
Token IDs: [7592, 2088]
```

The interface includes a RAM/Disk usage indicator in the top right corner and a navigation bar at the bottom showing "17 / 23".

16. Compare BERT and GPT-2 Tokenization



The image shows a Google Colab notebook titled "GEN-AI". The code cell contains the following Python code:

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")
gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

sentence = input("Enter sentence: ")

print("\nBERT Tokens:")
print(bert.tokenize(sentence))

print("\nGPT-2 Tokens:")
print(gpt2.tokenize(sentence))
```

The output of the code is:

```
Enter sentence: I Love GEN-AI

BERT Tokens:
['i', 'love', 'gen', '-', 'ai']

GPT-2 Tokens:
['I', 'Love', 'GEN', '-', 'AI']
```

The interface includes a RAM/Disk usage indicator in the top right corner and a navigation bar at the bottom showing "18 / 23".

17. Generate Contextual Embeddings

```
colab GEN-AI
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

sentence = input("Enter sentence: ")
inputs = tokenizer(sentence, return_tensors="pt")

with torch.no_grad():
    output = model(**inputs)

    print("Embeddings:")
    print(output.last_hidden_state)

Loading weights: 0% | 0/199 [00:00<?, ?it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----|-----|
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Enter sentence: Deekshiith is a good boy
Embeddings:
tensor([[[[-0.3605, -0.1715, -0.0329, ..., -0.2971, 0.7115, 0.3611],
          [ 0.5009, -1.1090, -0.0416, ..., 0.1776, 1.1123, -1.4815],
          [-0.0732, -1.2584, -0.0560, ..., 0.0030, 0.7256, -0.5527],
          ...,
          [ 0.2525, 0.0153, 0.5957, ..., -0.2476, 0.7950, -0.5587],
          [-0.1540, -0.6995, 0.0876, ..., 0.1193, 1.0125, -1.3842],
          [ 0.5780, -0.1140, -0.2496, ..., 0.2115, -0.2893, -0.4701]]]])

|< < 19/23 > >|
```

18. Compare Contextual Embeddings of Two Sentences

```
colab GEN-AI
from transformers import BertTokenizer, BertModel
import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertModel.from_pretrained("bert-base-uncased")

s1 = input("Enter first sentence: ")
s2 = input("Enter second sentence: ")

i1 = tokenizer(s1, return_tensors="pt")
i2 = tokenizer(s2, return_tensors="pt")

with torch.no_grad():
    o1 = model(**i1)
    o2 = model(**i2)

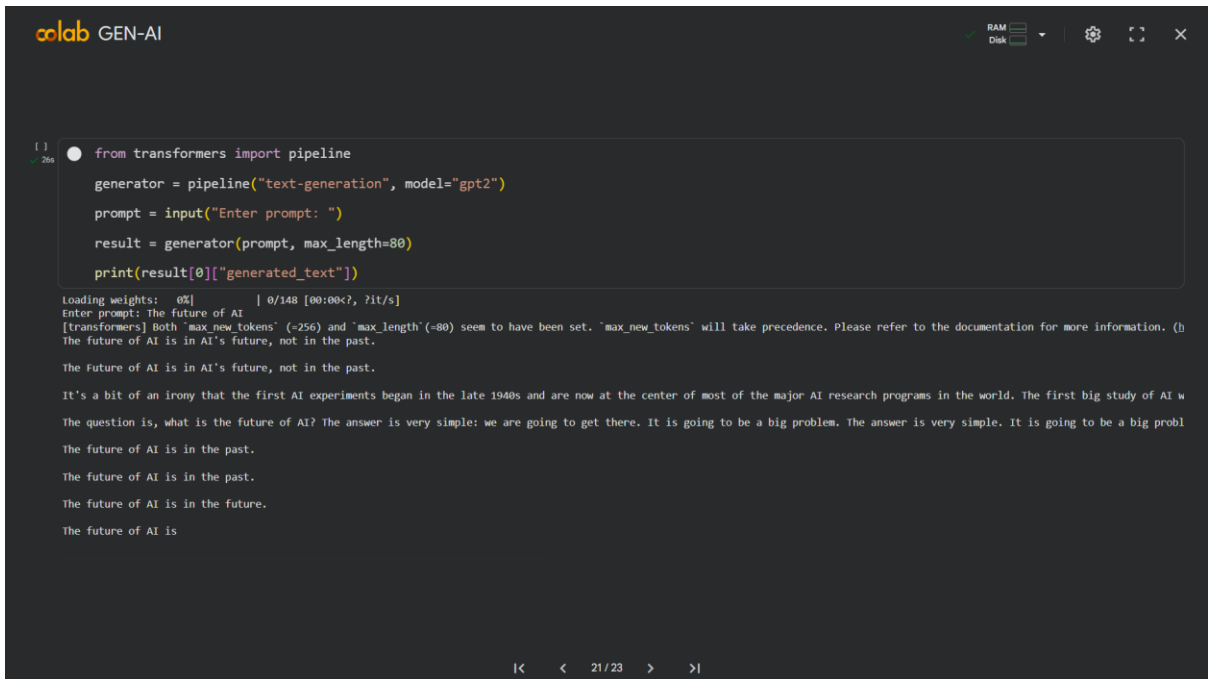
    print("Sentence 1 Shape:", o1.last_hidden_state.shape)
    print("Sentence 2 Shape:", o2.last_hidden_state.shape)

Loading weights: 0% | 0/199 [00:00<?, ?it/s]
[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key | Status |
-----|-----|
cls.predictions.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Enter first sentence: The cat is sleeping
Enter second sentence: The kitten is running
Sentence 1 Shape: torch.Size([1, 6, 768])
Sentence 2 Shape: torch.Size([1, 6, 768])

|< < 20/23 > >|
```

19. Generate Text from Prompt using GPT-2



The Colab GEN-AI interface displays a Jupyter notebook cell with Python code for GPT-2 text generation. The code imports the pipeline, sets the model to 'gpt2', and uses an input prompt. The output shows the generated text for the prompt 'The future of AI is in AI's future, not in the past.'.

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompt = input("Enter prompt: ")

result = generator(prompt, max_length=80)

print(result[0]["generated_text"])
```

Loading weights: 0% | 0/148 [00:00<?, ?it/s]
Enter prompt: The future of AI
[transformers] Both 'max_new_tokens' (~256) and 'max_length' (~80) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information. (h
The future of AI is in AI's future, not in the past.
The future of AI is in AI's future, not in the past.
It's a bit of an irony that the first AI experiments began in the late 1940s and are now at the center of most of the major AI research programs in the world. The first big study of AI w
The question is, what is the future of AI? The answer is very simple: we are going to get there. It is going to be a big problem. The answer is very simple. It is going to be a big probl
The future of AI is in the past.
The future of AI is in the past.
The future of AI is in the future.
The future of AI is

20. Compare Responses for Multiple Prompts



The Colab GEN-AI interface displays a Jupyter notebook cell with Python code for GPT-2 text generation. The code imports the pipeline, sets the model to 'gpt2', and uses an input prompt. The output shows the generated text for the prompt 'The future of AI is in AI's future, not in the past.'.

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

n = int(input("Enter number of prompts: "))

for i in range(n):
    prompt = input(f"Enter prompt {i+1}: ")
    result = generator(prompt, max_length=50)
    print(f"\nGenerated Response:")
    print(result[0]["generated_text"])
```

Loading weights: 0% | 0/148 [00:00<?, ?it/s]
Enter number of prompts: 2
Enter prompt 1: AI
[transformers] Both 'max_new_tokens' (~256) and 'max_length' (~50) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information.

Generated Response:
AI: Oh, I think so. And I'm going to go out and try to put together a great script.
JAVIER: And you've been working with a cast of people from all over the world, so you got to get to know each one of them.
STURGES: I think this is the season I would love to do with them. I think it all comes from their own experience and their own personal experience.
JAVIER: I am sure that there is so much more to do with them that you'll need to read through and figure out what you want to do.
STURGES: It's a question of when, and what, they're going to be. But as they say in the scene where they first go to the bar...
JAVIER: Wait, wait, I don't know.
STURGES: It was a long time ago.
JAVIER: You said in the scene, it's just another day.
STURGES: But I think it's so much more than that.
JAVIER: They are so passionate about something.
STURGES: It's so much more than just that.
Enter prompt 2: Space Exploration
[transformers] Both 'max_new_tokens' (~256) and 'max_length' (~50) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation for more information.
